CIT Campus Scheduling & Venue Management Platform

Architecture Consistency Review & Decision Register — v0.2

Date: 31 October 2026 **Status:** Engineering review; not institutional approval

Date: 31 October 2026

Status: Engineering review; not institutional approval

Scope: Master PRD v1.0 draft, Source Audit & Data Dictionary v0.1, ERD & PostgreSQL Schema Design v0.1, API Specification & Backend Domain Contracts v0.1, uploaded CrunchWeek starter pack and official `validator.py`.

0. Executive decision

Recommendation: proceed with a database-first modular monolith, but do not generate production migrations from ERD v0.1 yet.

The existing direction is sound: PostgreSQL as the system of record; a normalized relational model for operational data; immutable source and schedule versions; deterministic Python domain services; OR-Tools for optimization; and Gemini limited to interpretation and proposal preparation.

The current documents are broadly aligned, but v0.1 is not yet a migration-ready schema. The most important gaps are:

• The competition export contains `fest_week.session_overrides`, but the ERD does not define a first-class persistence model for those overrides.

• Dataset versioning, academic master data, schedule versioning and active/published pointers need sharper ownership and referential integrity.

• The official validator has several important behaviours and omissions that must be represented accurately; it is not a full institutional workflow engine.

• Some operational rules are only service-level today and need explicit transaction/locking designs.

• Analytics, notification and audit requirements need complete API and data ownership.

• UI is intentionally deferred; schema and API contracts must remain frontend-independent.

This review does not silently turn proposals into official CIT policy. Status labels below mean:

• **SOURCE-CONFIRMED:** present in the starter pack, problem statement or official validator.

• **PROPOSED:** recommended engineering design.

• **OPEN:** requires team, organiser or institutional confirmation.

• **NOT IN MVP:** consciously deferred.

---

1. Evidence reviewed

1.1 Source pack inspected

• `CrunchWeek_StarterPack/README.md`

• `data/cit_public.json`

• `examples/sample_disruption.json`

• `examples/schedule.example.json`

• `examples/sample_requests.md`

• `validator.py`

• Problem statement PDF and existing Master PRD draft.

The canonical dataset has these root keys:

`dataset_id`, `institute`, `calendar`, `priority_order`, `role_category_cap`, `rooms`, `room_unavailability`, `faculty`, `student_groups`, `courses`, `sessions`, `booking_requests`, `cancellations`, `exams`.

Observed public dataset counts: 27 rooms, 40 faculty, 14 student groups, 62 courses, 189 teaching sessions, 25 booking requests, 1 cancellation and 41 exams. These are dataset facts, not constants to hardcode.

1.2 Authority order

When sources disagree, use this order for the hackathon:

• Official problem statement and organiser clarifications.

• Uploaded starter-pack schema/examples and canonical JSON.

• Official `validator.py` for machine-checked acceptance behaviour.

• Approved project PRD and technical contracts.

• Engineering proposals in this review.

The validator explicitly says that if it and the app disagree, the validator wins.

---

2. Consistency matrix

Area	PRD	Data dictionary / source	ERD v0.1	API v0.1	Review
Canonical import	Required	Exact JSON contract	Dataset version + normalized records	Import endpoints	Aligned; add strict import schema and source checksum rules
Calendar	Required	Teaching, event and exam calendars differ	Calendar version tables	Configuration endpoint	Aligned; enforce calendar-scoped references
Timetable	Generate, view, adjust, publish	189 sessions and exact validator rules	Timetable versions and assignments	Generation, validation, views, manual assignment	Aligned conceptually; add occupancy projection and solver reproducibility
Spectra bookings	Parse, decide, explain	Exact request fields, caps, priority, export	Requests, decisions, allocations, approvals	Booking and approval endpoints	Missing explicit fest-week session override model
Exams	Schedule all exams	AM/PM, multi-room, half capacity	Exam and allocation tables	Exam schedule endpoints	Aligned; keep validator omission on duration explicit
Disruptions	Propose minimal movement; Admin approval	Five change types; permanent after application	Change sets and derived dataset	Disruption and proposal endpoints	Aligned; clarify atomic version activation
AI	Assistant proposes, backend validates	Raw request text and deterministic rules	Conversation/tool execution	Assistant endpoints	Aligned; enforce allowlisted typed tools
Campus Navigator	Indoor/outdoor 2D	No map assets in starter JSON	Navigation namespace	Map, place and route endpoints	Proposed future module; cannot claim source-backed campus geometry
Analytics	Utilisation, conflicts, pending approvals	Metrics derivable from records	Some metric tables	No dedicated analytics API group	API gap
Notifications	In-app notifications	Not in source pack	Notifications table	No notification endpoints	API gap
Audit	Explain who changed what and why	Source records have reasons/raw text	Audit logs	No audit read endpoints	API gap
Export	Exact JSON generated by app	Exact export shape	Export jobs/artifacts	Export endpoints	Add first-class `session_overrides` and golden-fixture round-trip tests

---

3. Verified validator contract — implementation must match

3.1 Timetable hard rules

The official validator checks:

• **H1:** every source session appears; unknown sessions/rooms and duplicate session IDs are violations.

• **H2:** no faculty overlap.

• **H3:** no student-group overlap. Sessions with the same non-null `basket_slot` use a shared occupancy key for the groups, allowing the parallel elective sessions to coexist.

• **H4:** no room overlap.

• **H5:** room capacity must meet session size.

• **H6:** room type must be in the session's allowed `room_types`.

• **H7:** faculty unavailability is respected.

• **H8:** room unavailability is respected.

• **H9:** teaching session fits P1–P9, does not cross lunch, and lab starts at a configured lab start period.

• **H10:** `needs_accessible` requires an accessible room.

• **H11:** all sessions sharing `basket_slot` use the same day and start period.

• **H12:** faculty daily teaching duration does not exceed that faculty member's `max_hours_per_day` (fallback 4 if absent).

• **H14:** fixed sessions retain exact day and start period.

The validator does not define H13 for timetable scheduling; H13 is used for booking allocation failures. Do not renumber or merge these rule identifiers.

3.2 Soft penalties

• **S1:** 2 points per occupied period overlapping an applicable faculty `avoid` preference.

• **S2:** 1 point for each unoccupied period between a group's first and last occupied teaching period in a day.

• **S3:** 3 points per group teaching hour above 6 in a day.

• **S4:** 3 points per additional same-course lecture for a group/day; basket course repetitions are separately counted.

• **S5:** 1 point per session whose `session.size / room.capacity` is below 0.4.

• **S6:** 1 point per session occupying P9.

Persist the component totals and explanatory evidence for each solver run. The objective is not merely “zero hard violations”; soft penalties and disruption stability also matter.

3.3 Booking validation details and limitations

The validator:

• flags duplicate decisions (B1);

• checks session overrides against an existing session and an approved booking that is allowed to displace a class (B2);

• rejects approved or alternative decisions for cancelled requests (B3);

• warns when a request has no decision; missing decision is not currently emitted as a hard violation;

• accepts only `approved`, `rejected`, `alternative_offered`;

• warns when rejection or alternative lacks a reason;

• checks room existence, event permission, type, capacity and requested features;

• requires an **approved** allocation to fit wholly inside one requested window;

• permits an `alternative_offered` allocation outside requested windows;

• checks room unavailability, timetable occupancy and mandatory-group timetable occupancy;

• calculates booking score and priority inversions.

Effective category is the lower-priority category after applying the requester's role cap. Category weights are 5/4/3/2 for institute/guest lecture/department/club event. Same-category ties use earlier `submitted_at`.

Important validator limitations (do not mistake these for product policy):

• It does not implement reviewer routing, authenticated identities, multi-stage approvals or no-self-approval.

• It does not check event booking intervals against the event-only periods; it checks period range P1–P13 and the room's event permission. The application should normally guide event requests to event periods, but must label any stricter rule as product policy unless organisers confirm it.

• It does not check two different bookings for overlapping mandatory student groups. The app should add this as a safety validation if bookings can target the same groups; record it as an application-level rule.

• It does not require a decision for every booking as a hard failure; it reports missing decisions as warnings. Product/export readiness should nevertheless require complete decisions.

• It does not enforce a requested room exactly; `requested_room` is a preference, not a hard constraint.

• An alternative offer is included in booking score at half weight when valid. Acceptance/decline is not represented in the export format.

• The validator does not implement the natural-language parser or interpret Hindi/Hinglish dates.

3.4 Exam validation details and limitations

The validator checks:

• each source exam appears exactly once;

• day and AM/PM session are valid;

• every allocated room exists and permits exams;

• effective seat count is `floor(room.capacity / 2)` per room;

• combined effective capacity meets exam size;

• a room and a student group cannot be assigned two exams in the same exam slot;

• each additional exam for a group on the same day adds 5 soft penalty points.

The source contains `duration_hours`, but the validator does not compare it to the selected AM/PM window duration. Preserve the field; do not silently invent a validator rule. If the product later enforces it, document it as an additional application policy and confirm with organisers.

3.5 Disruption contract

Supported change types are:

• `faculty_unavailable`

• `room_unavailable`

• `add_faculty`

• `add_session`

• `remove_session`

The validator applies changes to a copy of the dataset. The README says changes are permanent for every week after application. Preserve the original dataset and create a derived dataset version for an approved change set. Do not confuse a one-week Spectra `session_override` with a permanent disruption.

---

4. Required database architecture corrections

DB-01 — Make dataset versioning the root of imported academic data

Status: PROPOSED — required before migrations.

Every imported dataset is an immutable snapshot with:

• internal UUID;

• external `dataset_id`;

• source checksum and original filename;

• import schema version and application version;

• source JSON snapshot in object storage or immutable JSONB;

• import report and row counts;

• parent dataset version for derived datasets;

• lifecycle status and actor/timestamps.

Every imported academic record must be scoped to a dataset version (or be explicitly classified as a shared institute master). Avoid a mixture where some records are version-scoped and others accidentally point to the current active dataset.

Recommended identity pattern:

• Internal primary key: UUID.

• External source ID: preserved exactly, unique within the dataset version.

• Composite uniqueness: `(dataset_version_id, external_id)`.

• All operational records reference internal UUIDs and the exact dataset version used.

Do not use `cit-public-v1` itself as the only database primary key. Multiple imports of the same external dataset can exist with different checksums or import timestamps.

DB-02 — Separate source data from operational edits

Status: PROPOSED.

Imported source records should not be silently edited in place. Keep:

• immutable imported source version;

• approved derived dataset version for permanent disruptions;

• operational timetable, booking and exam versions;

• explicit current/published pointers.

A derived dataset should retain its parent and the applied change-set ID. Store both the raw change payload and normalized effect.

DB-03 — Add a first-class Spectra/fest-week version and session override model

Status: PROPOSED; required to meet export contract.

The export shape has:

• `fest_week.bookings[]`

• `fest_week.session_overrides[]`

The current ERD persists booking decisions and allocations but does not define a first-class table for the second collection. Add:

• `booking.fest_week_versions` (or an equivalent version record linked to dataset + baseline timetable);

• `booking.session_overrides` with booking decision, displaced session, new day, start period, room and reason;

• uniqueness preventing more than one effective override for a session in a fest-week version;

• FK to the baseline timetable assignment and the approved booking decision;

• immutable history when an override is changed.

The export adapter must build `session_overrides` from this table, never from ad hoc JSON assembled in the frontend.

DB-04 — Add explicit slot/period identity and preserve distinct calendars

Status: PROPOSED.

Teaching periods, event periods and exam sessions are different concepts. Keep separate calendar tables and stable identifiers:

• teaching/event period number belongs to a calendar version;

• exam session code belongs to an exam calendar version;

• booking allocation references an event period range;

• exam assignment references an exam session code;

• all time values are interpreted in the institute timezone.

Avoid one generic `period_id` shared across teaching P1–P9, event P10–P13 and exam AM/PM. A single API abstraction may be used, but the database must preserve domain type.

DB-05 — Add occupancy projections for concurrency-safe scheduling

Status: PROPOSED.

Solver validation alone is not enough when two users can make changes concurrently. Add a derived occupancy table or equivalent transaction-safe locking model for each timetable/fest/exam version.

For timetable occupancy, record resource type (`room`, `faculty`, `student_group`), resource key, day, period, assignment and version. Enforce uniqueness for exclusive occupancy. Elective basket occupancy needs a carefully defined group-occupancy key so parallel basket sessions are not incorrectly rejected, while all other sessions still conflict.

For bookings, reserve room-period occupancy in one transaction after locking affected room/time keys. Add mandatory-group checks in the same transaction. Never “check availability” and “insert allocation” as two unprotected operations.

Occupancy is a rebuildable projection, not the source of truth. Canonical assignments and allocations remain authoritative.

DB-06 — Model timetable and solver provenance

Status: PROPOSED.

Each timetable version should record:

• dataset version;

• parent/baseline timetable version where applicable;

• generation request/configuration;

• solver name and version;

• random seed where supported;

• time limit and worker count;

• objective weights/configuration version;

• run status, start/end, runtime;

• hard violation count and soft penalty components;

• official validator version/checksum and report artifact;

• who requested, approved and published it.

This is necessary to reproduce or explain a schedule. Do not store only `objective_value`.

DB-07 — Model booking decision history and workflow separately

Status: SOURCE-CONFIRMED export statuses; workflow PROPOSED/OPEN.

Keep request lifecycle separate from decision outcome. Preserve exact export values:

`approved`, `rejected`, `alternative_offered`.

Do not collapse `alternative_offered` into `approved`. If requesters can accept/decline alternatives, store that as a separate response lifecycle. Export semantics after acceptance remain OPEN until organiser confirmation.

Decision history should be append-only with an effective-decision pointer or version. Approval tasks and approval events remain separate. Enforce no-self-approval in the service and transaction layer, with tests; do not rely only on frontend hiding.

DB-08 — Make exam room allocations relational and version-scoped

Status: PROPOSED; validator requirements SOURCE-CONFIRMED.

One exam assignment can have multiple room allocation rows. Each assignment is unique per exam schedule version and exam. Enforce no duplicate room allocation and validate room/group slot exclusivity transactionally. Use the exact exam calendar version. Preserve source `duration_hours`, even though the validator currently does not compare it to AM/PM.

DB-09 — Add immutable audit events for every consequential action

Status: PRD requirement / PROPOSED implementation.

Audit at least:

• dataset import/activation;

• timetable generation, manual assignment, validation, approval and publication;

• booking submission, decision, alternative response and cancellation;

• disruption import, proposal, validation, approval and application;

• exam generation, approval and publication;

• map asset processing, review and publication;

• export generation and official validator result.

Record actor, action, target type/id, timestamp, reason, before/after reference or diff, request/correlation ID and relevant version IDs. Avoid copying secrets or unnecessary personal data into logs.

DB-10 — Use soft deletion only where the domain requires it

Status: PROPOSED.

Do not apply generic soft-delete to every table. Immutable imported source records and historical decisions should not be deleted. Use status/archival transitions for operational records. Use explicit removal changes for sessions as specified by the disruption contract. Hard-delete only ephemeral data under a retention policy.

---

5. API contract corrections and additions

5.1 Must add or clarify

API area	Required change	Reason
Dataset import	Return internal `dataset_version_id` separately from external `dataset_id`	Avoid identifier ambiguity
Dataset activation	Activation must reference exact immutable version UUID	Prevent “latest import” ambiguity
Timetable	Expose occupancy/validation details and solver provenance	Explain and reproduce output
Fest week	Add fest-week version endpoints and session-override create/list/validate operations	Current export requires overrides
Bookings	Include decision history, requester alternative response, and effective category explanation	Auditable decisions
Exams	Expose exam room allocations as child resources	Multi-room allocation
Disruptions	Apply endpoint must use baseline version precondition and atomic activation	Prevent stale proposals
Analytics	Add summary, room utilisation, conflict and pending-approval read endpoints	PRD requires dashboards
Notifications	Add inbox/read-state endpoints or mark notifications P1	PRD includes in-app notifications
Audit	Add scoped audit search/detail endpoints for Admin	PRD requires audit history
Exports	Return export checksum, exact source version IDs and validator report reference	Reproducibility

5.2 Suggested endpoint additions

• `GET /analytics/overview`

• `GET /analytics/room-utilisation`

• `GET /analytics/conflicts`

• `GET /analytics/booking-summary`

• `GET /notifications`

• `POST /notifications/{notification_id}/read`

• `GET /audit-events`

• `GET /fest-week-versions`

• `POST /fest-week-versions`

• `GET /fest-week-versions/{version_id}`

• `POST /fest-week-versions/{version_id}/session-overrides`

• `POST /fest-week-versions/{version_id}/validate`

These are proposed API names, not existing source endpoints. Keep the frontend out of the design decision: these contracts should be usable by a web UI, mobile client, CLI and AI assistant.

5.3 Concurrency and preconditions

Use:

• `Idempotency-Key` for job creation and external side-effect requests;

• `If-Match`/version token for edits against a known version;

• database transaction + row/advisory locks for booking and publication;

• stale baseline rejection for disruption apply;

• immutable versions rather than in-place mutation of published schedules.

A successful HTTP response must not claim a solver result or approval before the corresponding job/transaction has completed.

---

6. Recommended sustainable database design

6.1 PostgreSQL as the system of record

Use one PostgreSQL database initially, with logical schemas/namespaces:

• `core`

• `academic`

• `scheduling`

• `booking`

• `exams`

• `change_mgmt`

• `navigation`

• `assistant`

• `integration`

This is a modular monolith, not a microservice architecture. PostgreSQL constraints, transactions, foreign keys and indexes protect the data. Python domain services own business rules.

6.2 Normalized relational data + JSONB at the boundary

Use relational columns and join tables for data that must be queried, joined, constrained or versioned:

• rooms, faculty, student groups, courses, sessions;

• session-to-group and session-to-allowed-room-type relationships;

• bookings, booking windows, required features and mandatory groups;

• decisions, allocations and approvals;

• exams, exam courses, groups and room allocations;

• timetable assignments and occupancy.

Use JSONB for:

• exact imported source snapshot;

• raw disruption payloads whose shapes differ by change type;

• AI structured payloads and tool input/output snapshots;

• validator report payloads;

• non-query-critical metadata.

Do not store the entire live database as one JSONB document. Do not store all relationships as JSON arrays when relational constraints are required.

6.3 Internal IDs and external IDs

Use UUID primary keys internally. Keep original source identifiers in `external_id` columns scoped by dataset version. Every import adapter must preserve source IDs exactly and map them to internal UUIDs. Never derive identity from display names.

6.4 Constraints vs domain validation

Database constraints should enforce:

• primary and foreign keys;

• uniqueness within dataset/version;

• non-null and check constraints;

• one assignment per session per timetable version;

• one exam assignment per exam per exam schedule version;

• no duplicate allocation rows;

• one effective decision per booking request;

• valid parent-child version references.

Domain services and official validator should enforce complex rules:

• no faculty/group/room clashes;

• elective basket semantics;

• capacity/type/accessibility;

• lunch/lab/fixed-slot rules;

• priority inversions;

• exam half-capacity;

• minimal movement;

• map graph connectivity and physical plausibility.

Use occupancy projections and transaction locks to close race conditions that pure application validation cannot prevent.

6.5 Version immutability

Once a timetable, exam schedule, fest-week plan or map version is approved/published, do not edit its contents in place. Create a new version with a parent reference. Keep the currently published pointer separate from historical versions. Rollback should move the pointer to a previously approved immutable version, with an audit event.

---

7. Backend architecture and implementation order

7.1 Recommended stack (still conditional on team skills)

• Python 3.11+ / FastAPI / Pydantic.

• PostgreSQL.

• SQLAlchemy 2.x and Alembic.

• OR-Tools CP-SAT for timetable/exam/repair optimization.

• Gemini structured output/function calling for constrained parsing and assistance.

• A worker process for solver and map processing jobs.

• Pytest and the official `validator.py`.

• React + TypeScript frontend later; no UI decisions are required to finish the domain model.

Redis/Celery is optional. For a small hackathon deployment, a database-backed job table and a separate worker process may be sufficient. Add Redis only if the team needs it.

Payload CMS remains an alternative if team capability strongly favours TypeScript, but it would likely leave the Python solver as a separate service. Do not choose architecture by trend; choose based on who can implement, test and debug it during the hackathon.

7.2 Build sequence — database/backend before UI

Phase A — Contract and schema freeze

• Resolve the OPEN decisions listed below.

• Approve source-to-relational mapping.

• Update ERD with fest-week overrides, analytics/notifications/audit and occupancy.

• Define migration naming, versioning and seed strategy.

• Create schema diagram and migration review checklist.

Phase B — Database foundation

• Create PostgreSQL schemas and extensions (PostGIS only when Navigator work begins).

• Create core institute, dataset version and calendar tables.

• Create academic master tables and all relationship tables.

• Add import job, source artifact and audit tables.

• Add constraints and indexes.

• Add Alembic migrations and migration rollback tests.

Phase C — Import and deterministic validation

• Build strict Pydantic models for the canonical JSON.

• Validate source file before opening a DB transaction.

• Import into a new immutable dataset version in a transaction.

• Generate an import report and compare all source counts.

• Re-export canonical source fields in a round-trip test where possible.

• Run the official validator against golden fixtures.

Phase D — Scheduling and bookings

• Implement timetable version and assignment persistence.

• Implement deterministic hard validator.

• Implement OR-Tools solver and persist provenance/metrics.

• Implement booking requests, decision history, allocations and priority evaluation.

• Implement fest-week version and session overrides.

• Implement exam schedule and multi-room allocation.

• Implement disruption-derived dataset versions and minimal-movement repair.

Phase E — API, AI and integration

• Expose version-aware APIs.

• Add AI tools as narrow backend functions.

• Every AI proposal passes deterministic validation.

• Generate exports from persisted state only.

• Run official validator and store the report/checksum.

Phase F — UI/UX later

The frontend should be built after domain/API contracts stabilize. It must not own validation, priority calculation, solver rules, approval authority or export serialization.

---

8. Open decisions — do not silently guess

ID	Question	Recommended default	Status
O-01	Can team members implement Python/FastAPI confidently?	Use FastAPI if yes; otherwise explicitly choose a TypeScript backend plus Python solver boundary	OPEN
O-02	Who reviews each booking category and department?	Admin-controlled routing with scoped reviewer rules	OPEN
O-03	Can one request require multiple approval stages?	Support approval-task stages in schema; use one stage for MVP unless policy says otherwise	OPEN
O-04	What happens after an alternative is accepted/declined?	Store response separately; confirm export status semantics with organiser	OPEN
O-05	Can booking requests reserve multiple rooms?	No for competition export; defer	NOT IN MVP
O-06	Are event bookings restricted to P10–P13 by policy?	UI default to event periods; do not assert validator enforces this	OPEN
O-07	Should separate bookings conflict when they require the same student group?	Yes, add application-level group occupancy check	PROPOSED
O-08	Who may manually edit timetable assignments?	Role-scoped proposal/edit permissions; Admin final publish	OPEN
O-09	Do generated timetables require Admin approval before publish?	Yes, proposed gate	OPEN
O-10	Are student accounts required for hackathon demo?	Read-only timetable/place access can be public/demo mode; no fifth competition role	OPEN
O-11	What is the recurrence behaviour for `series_id`?	Preserve field; no recurrence engine until clarified	OPEN
O-12	What indoor/outdoor map assets and coordinate references are available?	No map is authoritative until Admin review	OPEN
O-13	Does the organiser expect every booking to have a decision even though validator only warns?	Yes, require complete decisions for application export readiness	OPEN / recommended
O-14	Should exam duration be enforced against AM/PM duration?	Preserve source field; ask organiser before adding stricter validation	OPEN

---

9. Explicit corrections to v0.1 documents

• **ERD:** add a fest-week version and `session_overrides` persistence model.

• **ERD:** add an occupancy projection/locking design and define elective basket occupancy keys.

• **ERD:** clarify that every operational record references exact dataset/timetable/calendar versions.

• **API:** distinguish internal UUIDs from external `dataset_id`.

• **API:** add analytics, notification and audit endpoint groups or move these features out of the current release explicitly.

• **API:** add fest-week override endpoints and contracts.

• **API/DB:** clarify alternative acceptance lifecycle separately from competition decision status.

• **Validator docs:** explicitly state validator limitations listed in Section 3; do not claim it enforces reviewer workflow, event-only booking periods, mandatory-group booking-to-booking conflicts or exam duration matching.

• **Export:** require the application to produce every booking decision for a ready-to-submit export, even though missing decisions are only warnings in the current validator.

• **Implementation:** do not create migrations directly from v0.1. Approve the revised schema first.

---

10. Acceptance gate before writing migrations

The database design is ready for migration work only when all checks below pass:

• [ ] Source JSON root keys and nested fields have a complete import mapping.

• [ ] Every source ID is preserved and scoped to the imported dataset version.

• [ ] Calendar, event and exam time models remain separate.

• [ ] Dataset, timetable, exam, fest-week and map versions have clear parent/current/published semantics.

• [ ] Every API write operation maps to a table, transaction and authorization rule.

• [ ] Every persisted entity has an owner module and deletion/retention policy.

• [ ] Fest-week session overrides are persisted relationally.

• [ ] Booking decision history and alternative response are distinct.

• [ ] Occupancy/concurrency strategy is testable.

• [ ] Official validator behaviours and omissions are documented.

• [ ] Export JSON can be generated from database state and matches golden fixtures.

• [ ] Migration, rollback, import idempotency and version activation tests are defined.

• [ ] Team confirms language/tool capability and MVP scope.

Current gate result: NOT READY FOR PRODUCTION MIGRATIONS. The architecture direction is approved as a working proposal, but the schema needs the corrections above and the OPEN policy/team decisions must be resolved or explicitly deferred.

11. Next deliverable

Milestone 2D — Revised Database Blueprint v0.2.

This should replace the conceptual gaps in ERD v0.1 with:

• revised entity inventory and relationships;

• explicit dataset/version ownership;

• fest-week/session-override model;

• occupancy model;

• full source-to-table mapping;

• constraints, indexes and transaction boundaries;

• migration sequence;

• PostgreSQL DDL outline (not yet deployed);

• schema acceptance tests.

Only after Milestone 2D should we generate Alembic migrations and begin the backend implementation. UI/UX remains deliberately later.